CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT – 8

Name: Hari **Krishan** **UID:** 24BAI70898

Section/Group: 24AIT_NTPP 3 (A) **Branch/Semester:** CSE_AIML (CS221) / 5th

Subject Code: 24 CSP-337 **Subject Name:** Full Stack - II

Experiment – 8.1 and 8.2

1) AIM: To implement reliability mechanisms in an event-driven system using retry strategies, dead-letter queues (DLQ) and idempotency patterns.

2) OBJECTIVE

1. To understand failure handling in distributed systems
2. To implement retry mechanisms with exponential backoff
3. To configure dead-letter queues for failed events
4. To ensure idempotent processing of messages

3) SOFTWARE REQUIREMENT

1. Java(JDK 17+)
2. Spring Boot
3. IDE(VSCode/Eclipse)
4. Postman/API testing tool
5. Mysql workbench
6. Docker
7. Apache Kafka

4) THEORY

In distributed systems, failures such as network issues, service downtime, or processing errors are inevitable. To ensure system reliability, mechanisms such as retries, dead-letter queues, and idempotency are implemented.

A **retry mechanism** attempts to process a failed message again, often using **exponential back off** to avoid overwhelming the system.

A **Dead Letter Queue (DLQ)** stores messages that fail after multiple retries. This prevents system blockage and allows developers to analyze and fix issues later.

Kafka guarantees **at-least-once delivery**, meaning a message may be delivered more than once. To prevent duplicate processing, **idempotency** is implemented, ensuring that processing the same message multiple times does not cause inconsistent results.

These mechanisms are essential for building **fault-tolerant, reliable, and production-ready distributed systems**.

5) IMPLEMENTATION/PROCEDURE

1. Start by setting up a Docker and configuring Kafka.
2. Build the spring boot application using maven, dependencies and suitable java version
3. Create a database in MySQL to handle data.
4. Ensure the spring boot project is connected to the kafka through the compose.yaml file. 5. Implement retry mechanism with exponential back off
6. Configure DLQ topic for failed messages.

7. Route events to DLQ after retries.
8. Implement idempotency using event tracking.
9. Ensure the application, kafka and batabases are running.
10. Test failure scenarios and verify system behavior.

6) CODE `compose.yaml` starts a single-node Kafka broker

```
services:  kafka:    image:
apache/kafka:latest
container_name: reliability-kafka
ports:
    - "9092:9092"
environment:
    KAFKA_NODE_ID: 1
    KAFKA_PROCESS_ROLES: broker,controller
    KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
    KAFKA_ADVERTISED_LISTENERS:  PLAINTEXT://localhost:9092
KAFKA_CONTROLLER_LISTENER_NAMES:            CONTROLLER
KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
    KAFKA_CONTROLLER_QUORUM_VOTERS: 1@localhost:9093
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
    KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
    KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
```

Configure the applications.properties

```
spring.application.name=kafka-reliability
server.port=8080 # MySQL configuration
spring.datasource.url=jdbc:mysql://localhost:3306/kafka_reliability
spring.datasource.username=root spring.datasource.password=mypassword
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver spring.sql.init.mode=always

# Kafka configuration spring.kafka.bootstrap-servers=localhost:9092
spring.kafka.consumer.group-id=order-group
spring.kafka.consumer.auto-offset-reset=earliest
spring.kafka.consumer.key-deserializer=org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
spring.kafka.consumer.value-deserializer=org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
spring.kafka.consumer.properties[spring.deserializer.key.delegate.class]=org.apache.kafka.common.serialization.Str
ingDeserializer
spring.kafka.consumer.properties[spring.deserializer.value.delegate.class]=org.springframework.kafka.support.serial
izer.JacksonJsonDeserializer
spring.kafka.consumer.properties[spring.json.trusted.packages]=com.example.kafka_reliability
spring.kafka.consumer.properties[spring.json.value.default.type]=com.example.kafka_reliability.OrderEvent
spring.kafka.consumer.properties[spring.json.use.type.headers]=false
# Producer configuration
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer
spring.kafka.producer.value-serializer=org.springframework.kafka.support.serializer.JacksonJsonSerializer
```

create scema.sql to define tables(orders,proceesed_events and event_tracking)

```
CREATE TABLE IF NOT EXISTS processed_events (     event_id VARCHAR(100) NOT
NULL PRIMARY KEY,
   processed_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
```

```
);
CREATE TABLE IF NOT EXISTS orders (     order_id
VARCHAR(100) NOT NULL PRIMARY KEY,     event_id
VARCHAR(100) NOT NULL UNIQUE,     status
VARCHAR(50) NOT NULL,
   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
--to store each event's progress separately, so that we can retry failed events without affecting others
CREATE TABLE IF NOT EXISTS event_tracking (
event_id VARCHAR(100) NOT NULL PRIMARY KEY,
order_id VARCHAR(100) NOT NULL,     status
VARCHAR(50) NOT NULL,
   attempt_count INT NOT NULL DEFAULT 0,
last_error TEXT,
   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
      ON UPDATE CURRENT_TIMESTAMP
);
```

Define the event format in `OrderEvent.java`(it is the message sent through Kafka)

```
package com.example.kafka_reliability;

public class OrderEvent {
private String eventId;
private String orderId;     private
String status;
   private boolean shouldFail;

   public OrderEvent() {
   }

   public OrderEvent(
String eventId,
         String          orderId,
String status,           boolean
shouldFail) {      this.eventId
= eventId;       this.orderId =
orderId;           this.status =
status;
      this.shouldFail = shouldFail;
   }

   public String getEventId() {
      return eventId;
   }
   public void setEventId(String eventId) {
      this.eventId = eventId;
   }
   public String getOrderId() {
      return orderId;
   }
```

```
    public void setOrderId(String orderId) {
      this.orderId = orderId;
    }
    public String getStatus() {
       return status;
    }
    public void setStatus(String status) {
       this.status = status;
    }
    public boolean isShouldFail() {
       return shouldFail;
    }
    public void setShouldFail(boolean shouldFail) {
this.shouldFail = shouldFail;
    }
}
```

Publish events through the REST API. Create `OrderProducerController.java` to handle `POST /api/orders` package

```
com.example.kafka_reliability.controller;

import com.example.kafka_reliability.OrderEvent;

import org.springframework.http.HttpStatus; import
org.springframework.http.ResponseEntity; import
org.springframework.kafka.core.KafkaTemplate; import
org.springframework.web.bind.annotation.*; import
java.util.UUID;
import java.util.concurrent.CompletableFuture;
import com.example.kafka_reliability.service.EventTrackingService;

@RestController
@RequestMapping("/api/orders") public class
OrderProducerController {     private final
KafkaTemplate<Object, Object> kafkaTemplate;     private final
EventTrackingService eventTrackingService;

    public OrderProducerController(
          KafkaTemplate<Object, Object> kafkaTemplate,
EventTrackingService        eventTrackingService)        {
this.kafkaTemplate = kafkaTemplate;
       this.eventTrackingService = eventTrackingService;
    }

    @PostMapping
    public CompletableFuture<ResponseEntity<String>> publish(
@RequestBody OrderEvent event) {

       if (event == null) {
          return CompletableFuture.completedFuture(
```

```
                ResponseEntity.badRequest().body("Order event is required")
            );
        }
        if (event.getOrderId() == null || event.getOrderId().isBlank()) {
    return CompletableFuture.completedFuture(
                ResponseEntity.badRequest().body("Order ID is required")
            );
        }

        if (event.getStatus() == null || event.getStatus().isBlank()) {
            return CompletableFuture.completedFuture(
                ResponseEntity.badRequest().body("Order status is required")
            );
        }

        if (event.getEventId() == null
              || event.getEventId().isBlank()) {
            event.setEventId(UUID.randomUUID().toString());
        }
        eventTrackingService.markPublishing(event);

        return kafkaTemplate.send(
                    "orders",
                    event.getEventId(),
                    event
                )
                .handle((result, error) -> {
                    if (error != null) {
                        String message = error.getMessage() == null
                                ? error.toString()
                                : error.getMessage();

                        eventTrackingService.markPublishFailed(
                                event.getEventId(),
message
                        );

                        return ResponseEntity
                                .status(HttpStatus.SERVICE_UNAVAILABLE)
                                .body("Could not confirm event publication. ID: "
                                        + event.getEventId());
                    }
                    eventTrackingService.markPublished(event);
                    return ResponseEntity
                            .accepted()
                            .body("Event published. ID: "
                                    + event.getEventId());
                });
        }
}
```

Create `OrderConsumer.java` to listen to `orders` and passes each deserialized event to the processing service

```
package com.example.kafka_reliability.consumer;
import com.example.kafka_reliability.OrderEvent;
import com.example.kafka_reliability.service.OrderProcessingService;
import org.springframework.kafka.annotation.KafkaListener; import
org.springframework.stereotype.Service;

@Service
public class OrderConsumer {
   private final OrderProcessingService processingService;

   public OrderConsumer(
         OrderProcessingService processingService) {
this.processingService = processingService;
   }

   @KafkaListener(
         topics = "orders",
         groupId = "order-group"
   )
   public void consume(OrderEvent event) {

      System.out.println(
            "Received event: " + event.getEventId());

      String result = processingService.processOrder(event);

      if ("DUPLICATE".equals(result)) {
         System.out.println(
               "Duplicate ignored: " + event.getEventId());
return;
      }

      System.out.println(
            "Order processed successfully: "
                  + event.getOrderId());
   }
}
```

`OrderProcessingService.java` validates the event, claims its ID in `processed_events`, and inserts the order. The ID claim and order insert are performed in a database transaction.

```
package com.example.kafka_reliability.service;

import com.example.kafka_reliability.OrderEvent; import
org.springframework.jdbc.core.JdbcTemplate; import
org.springframework.stereotype.Service; import
org.springframework.transaction.annotation.Transactional; //import
com.example.kafka_reliability.service.EventTrackingService;

@Service
```

```
public class OrderProcessingService {
private final JdbcTemplate jdbcTemplate;
    private final EventTrackingService eventTrackingService;
    public OrderProcessingService(
    JdbcTemplate jdbcTemplate,
          EventTrackingService eventTrackingService) {
    this.jdbcTemplate = jdbcTemplate;
       this.eventTrackingService = eventTrackingService;
    }

    @Transactional
    public String processOrder(OrderEvent event) {

       if (event.getEventId() == null              ||
event.getEventId().isBlank()) {            throw
new IllegalArgumentException(
                "Event ID is required");
       }

       if (event.getOrderId() == null              ||
event.getOrderId().isBlank()) {            throw
new IllegalArgumentException(
                "Order ID is required");
       }

       if (event.getStatus() == null               ||
event.getStatus().isBlank()) {         throw new
IllegalArgumentException(
                "Order status is required");
       }

       // Atomically claim this event ID.
       // A previously processed event produces zero inserted rows.
int inserted = jdbcTemplate.update(
             """
             INSERT IGNORE INTO processed_events
                (event_id, processed_at)
             VALUES (?, CURRENT_TIMESTAMP)
             """,
             event.getEventId()
       );

       if (inserted == 0) {
          return "DUPLICATE";
       }

       // Simulate a processing failure to test retry and rollback.
       if    (event.isShouldFail())    {                  throw    new
       RuntimeException(
                "Simulated order processing failure"); }
```

```
        // Perform the business operation in the same transaction. jdbcTemplate.update(
            """
            INSERT INTO orders (order_id, event_id, status)
            VALUES (?, ?, ?)
            """,
            event.getOrderId(),
event.getEventId(),
            event.getStatus()
        );
        eventTrackingService.markProcessed(event);

        return "PROCESSED";
    }
}
```

`KafkaConfig.java` declares the `orders` and `orders.DLT` topics and configures exponential backoff retries. After retries are exhausted, the recoverer sends the failed record to `orders.DLT`.

```
package com.example.kafka_reliability;

import org.apache.kafka.clients.admin.NewTopic; import
org.apache.kafka.common.TopicPartition; import
org.springframework.context.annotation.Bean; import
org.springframework.context.annotation.Configuration; import
org.springframework.kafka.config.TopicBuilder; import
org.springframework.kafka.core.KafkaTemplate;
import org.springframework.kafka.listener.DeadLetterPublishingRecoverer; import
org.springframework.kafka.listener.DefaultErrorHandler;
import org.springframework.kafka.support.ExponentialBackOffWithMaxRetries;
import org.springframework.kafka.listener.ConsumerRecordRecoverer; import
com.example.kafka_reliability.service.EventTrackingService; import
org.springframework.kafka.listener.RetryListener;

@Configuration
public class KafkaConfig {
    @Bean
    public NewTopic ordersTopic() {
        return TopicBuilder.name("orders")
            .partitions(1)
            .replicas(1)
            .build();
    }

    @Bean     public NewTopic
deadLetterTopic() {
        return TopicBuilder.name("orders.DLT")
            .partitions(1)
            .replicas(1)
            .build();
    }
```

```
    @Bean
    public DefaultErrorHandler errorHandler(
    KafkaTemplate<Object, Object> kafkaTemplate,
            EventTrackingService eventTrackingService) {
    DeadLetterPublishingRecoverer recoverer =
                new DeadLetterPublishingRecoverer(
kafkaTemplate,                      (record,
exception) ->                           new
TopicPartition(
"orders.DLT",
record.partition()
                            )
                );
                recoverer.setFailIfSendResultIsError(true);

                ExponentialBackOffWithMaxRetries backOff =
                new ExponentialBackOffWithMaxRetries(3);

                backOff.setInitialInterval(1000L);
backOff.setMultiplier(2.0);
                backOff.setMaxInterval(4000L);

                ConsumerRecordRecoverer trackingRecoverer = (record, exception) -> {
                   recoverer.accept(record, exception);
if (record.key() instanceof String eventId) {
                      eventTrackingService.markDeadLettered(
                            eventId,
                            exception.getMessage() == null
? exception.toString()
                                  : exception.getMessage()
                      );
                   }
                };
                DefaultErrorHandler errorHandler =                          new
DefaultErrorHandler(trackingRecoverer,                          backOff);
errorHandler.setRetryListeners(new RetryListener() {
                   @Override
public void failedDelivery(
                      org.apache.kafka.clients.consumer.ConsumerRecord<?, ?> record,
Exception exception,                        int deliveryAttempt) {
                      if (record.key() instanceof String eventId) {
                         eventTrackingService.recordAttempt(
                               eventId,
deliveryAttempt,
                               exception.getMessage() == null
                                     ? exception.toString()
                                     : exception.getMessage()
                         );
                      }
                   } }); return
                errorHandler; }
```

```
}
```

`EventTrackingService.java` writes event progress to the MySQL `event_tracking` table. Other backend components call it to record statuses such as `PUBLISHING`, `PUBLISHED`, `PUBLISH_FAILED`, and `PROCESSED`, plus retry attempts and the last error. The Kafka error handler also uses it to record `DEAD_LETTERED`

```
package com.example.kafka_reliability.service;

import com.example.kafka_reliability.OrderEvent; import
org.springframework.jdbc.core.JdbcTemplate; import
org.springframework.stereotype.Service;

@Service
public class EventTrackingService {

   private final JdbcTemplate jdbcTemplate;

   public EventTrackingService(JdbcTemplate jdbcTemplate) {
this.jdbcTemplate = jdbcTemplate;
   }

   public void markPublishing(OrderEvent event) {
      jdbcTemplate.update(
            """
            INSERT INTO event_tracking (event_id, order_id, status)
            VALUES (?, ?, 'PUBLISHING')
            ON DUPLICATE KEY UPDATE
order_id        =        VALUES(order_id),
status = CASE
                  WHEN status IN ('PROCESSED', 'DUPLICATE', 'DEAD_LETTERED')
                     THEN status
                  ELSE 'PUBLISHING'
               END,
               updated_at = CURRENT_TIMESTAMP
            """,
            event.getEventId(),
            event.getOrderId()
      );
   }

   public void markPublished(OrderEvent event) {
   jdbcTemplate.update(
            """
            INSERT INTO event_tracking (event_id, order_id, status)
            VALUES (?, ?, 'PUBLISHED') ON
            DUPLICATE KEY UPDATE
               order_id = VALUES(order_id),
               status = CASE
                  WHEN status IN ('PROCESSED', 'DUPLICATE', 'DEAD_LETTERED')
                     THEN status
```

```
                    ELSE 'PUBLISHED'
                END,
                updated_at = CURRENT_TIMESTAMP
            """,
event.getEventId(),
            event.getOrderId()
        );
    }

    public void markPublishFailed(String eventId, String error) {
jdbcTemplate.update(
            """
            UPDATE event_tracking
            SET status = CASE
                WHEN status IN ('PROCESSED', 'DUPLICATE', 'DEAD_LETTERED')
                  THEN status
                ELSE 'PUBLISH_FAILED'
              END,
              last_error = ?,
              updated_at = CURRENT_TIMESTAMP
WHERE event_id = ?
            """,
error,
eventId
        );
    }

    public void markProcessed(OrderEvent event) {
        jdbcTemplate.update(
            """
            INSERT INTO event_tracking (event_id, order_id, status)
            VALUES  (?,  ?,  'PROCESSED')
ON     DUPLICATE     KEY     UPDATE
order_id        =        VALUES(order_id),
status = 'PROCESSED',
                last_error = NULL,
              updated_at = CURRENT_TIMESTAMP
            """,
            event.getEventId(),
            event.getOrderId()
        );
    }

    public void recordAttempt(String eventId, int attempt, String error) {
        jdbcTemplate.update(
            """
            UPDATE event_tracking
            SET attempt_count = GREATEST(attempt_count, ?),
              last_error = ?,
              updated_at = CURRENT_TIMESTAMP WHERE
            event_id = ?
```

```
                """,
            attempt,
            error,
                eventId
        );
    }

    public void markDeadLettered(String eventId, String error) {
jdbcTemplate.update(
                """
                UPDATE event_tracking
                SET status = 'DEAD_LETTERED',
                    last_error = ?,
                    updated_at = CURRENT_TIMESTAMP
WHERE event_id = ?
                 AND status NOT IN ('PROCESSED', 'DUPLICATE')
                """,
error,
eventId
        );
    }
}
```

`EventTrackingController.java` provides read APIs for tracking data

```
package com.example.kafka_reliability.controller;

import com.example.kafka_reliability.dto.PageResponse; import
org.springframework.http.HttpStatus; import
org.springframework.jdbc.core.JdbcTemplate; import
org.springframework.web.bind.annotation.GetMapping; import
org.springframework.web.bind.annotation.PathVariable; import
org.springframework.web.bind.annotation.RequestMapping; import
org.springframework.web.bind.annotation.RequestParam; import
org.springframework.web.bind.annotation.RestController; import
org.springframework.web.server.ResponseStatusException;

import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/events")
public class EventTrackingController {

    private static final int MAX_PAGE_SIZE = 100;
    private final JdbcTemplate jdbcTemplate;

    public EventTrackingController(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate; }

    @GetMapping
    public PageResponse<Map<String, Object>> listEvents(
```

```
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "20") int size,
            @RequestParam(required = false) String status) {
        validatePage(page, size);

        String normalizedStatus = normalizeStatus(status);
        String filter = normalizedStatus == null ? "" : " WHERE status = ?";
        Object[] filterArgs = normalizedStatus == null
                ? new Object[0]
                : new Object[]{normalizedStatus};

        Long total = jdbcTemplate.queryForObject(
                "SELECT COUNT(*) FROM event_tracking" + filter,
                Long.class,
filterArgs
        );
        List<Object> queryArgs = new java.util.ArrayList<>(
                java.util.Arrays.asList(filterArgs));
queryArgs.add(size);
        queryArgs.add((long) page * size);

        List<Map<String, Object>> events = jdbcTemplate.queryForList(
                """
                SELECT event_id, order_id, status, attempt_count,
                     last_error, created_at, updated_at
                FROM event_tracking
                """
                     + filter
                     + " ORDER BY created_at DESC LIMIT ? OFFSET ?",
queryArgs.toArray()
        );

        return PageResponse.of(events, page, size, total == null ? 0 : total);
    }

    @GetMapping("/{eventId}")     public Map<String, Object>
getEvent(@PathVariable String eventId) {
        List<Map<String, Object>> events = jdbcTemplate.queryForList(
                """
                SELECT event_id, order_id, status, attempt_count,
                     last_error, created_at, updated_at
FROM event_tracking                WHERE
event_id = ?
                """,
 eventId
        );
        if (events.isEmpty()) {
            throw new ResponseStatusException(HttpStatus.NOT_FOUND, "Event not found");
        }
        return events.get(0);
    }
```

```
    private static void validatePage(int page, int size) {
    if (page < 0 || size < 1 || size > MAX_PAGE_SIZE) {
            throw new ResponseStatusException(
    HttpStatus.BAD_REQUEST,
                    "page must be non-negative and size must be between 1 and 100"
            );
        }
    }

    private static String normalizeStatus(String status) {
        if (status == null || status.isBlank()) {
return null;
        }
        return status.trim().toUpperCase(java.util.Locale.ROOT);
    }
}
```

`OrderQueryController.java` reads successfully stored orders from the `orders` database table:

```
package com.example.kafka_reliability.controller;

import com.example.kafka_reliability.dto.PageResponse; import
org.springframework.http.HttpStatus; import
org.springframework.jdbc.core.JdbcTemplate; import
org.springframework.web.bind.annotation.GetMapping; import
org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;

import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/orders")
public class OrderQueryController {

    private static final int MAX_PAGE_SIZE = 100;
private final JdbcTemplate jdbcTemplate;

    public OrderQueryController(JdbcTemplate jdbcTemplate) {
this.jdbcTemplate = jdbcTemplate;
    }

    @GetMapping
    public PageResponse<Map<String, Object>> listOrders(
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "20") int size,
@RequestParam(required = false) String status) {        if
(page < 0 || size < 1 || size > MAX_PAGE_SIZE) {
throw new ResponseStatusException(
```

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

```
                HttpStatus.BAD_REQUEST,
                "page must be non-negative and
size must be between 1 and 100"
            );
        }

        String normalizedStatus = status == null || status.isBlank()
            ? null
            : status.trim();
        String filter = normalizedStatus == null ? "" : " WHERE status = ?";
        Object[] filterArgs = normalizedStatus == null
            ? new Object[0]
            : new Object[]{normalizedStatus};

        Long total = jdbcTemplate.queryForObject(
            "SELECT COUNT(*) FROM orders" + filter,
            Long.class,
    filterArgs      );
        List<Object> queryArgs = new java.util.ArrayList<>(
            java.util.Arrays.asList(filterArgs));
        queryArgs.add(size);
        queryArgs.add((long) page * size);

        List<Map<String, Object>> orders = jdbcTemplate.queryForList(
            """
            SELECT order_id, event_id, status, created_at
            FROM orders
            """
                + filter
                + " ORDER BY created_at DESC LIMIT ? OFFSET ?",
    queryArgs.toArray()
        );

        return PageResponse.of(orders, page, size, total == null ? 0 : total);
    }
}
```

records

`DeadLetterEventController.java` returns paginated tracking `DEAD_LETTERED`

```
package com.example.kafka_reliability.controller;

import com.example.kafka_reliability.dto.PageResponse; import
org.springframework.http.HttpStatus; import
org.springframework.jdbc.core.JdbcTemplate; import
org.springframework.web.bind.annotation.GetMapping; import
```

whose status is

```
org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;
```

```
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/dead-letter-events")
public class DeadLetterEventController {

   private static final int MAX_PAGE_SIZE = 100;
   private final JdbcTemplate jdbcTemplate;

   public DeadLetterEventController(JdbcTemplate jdbcTemplate) {
this.jdbcTemplate = jdbcTemplate;
   }

   @GetMapping
   public PageResponse<Map<String, Object>> listDeadLetterEvents(
         @RequestParam(defaultValue = "0") int page,
@RequestParam(defaultValue = "20") int size) {         if
(page < 0 || size < 1 || size > MAX_PAGE_SIZE) {
throw new ResponseStatusException(
               HttpStatus.BAD_REQUEST,
               "page must be non-negative and size must be between 1 and 100"
         );
      }

      Long total = jdbcTemplate.queryForObject(
            "SELECT COUNT(*) FROM event_tracking WHERE status = 'DEAD_LETTERED'",
            Long.class
      );
      List<Map<String, Object>> deadLetterEvents = jdbcTemplate.queryForList(
            """
            SELECT event_id, order_id, status, attempt_count,
                  last_error, created_at, updated_at
            FROM event_tracking
            WHERE status = 'DEAD_LETTERED'
ORDER BY updated_at DESC               LIMIT
? OFFSET ?
            """,
            size,
            (long) page * size
      );
      return PageResponse.of(
deadLetterEvents,
            page,
size,
            total == null ? 0 : total
      );
```

}

}

7) OUTPUT

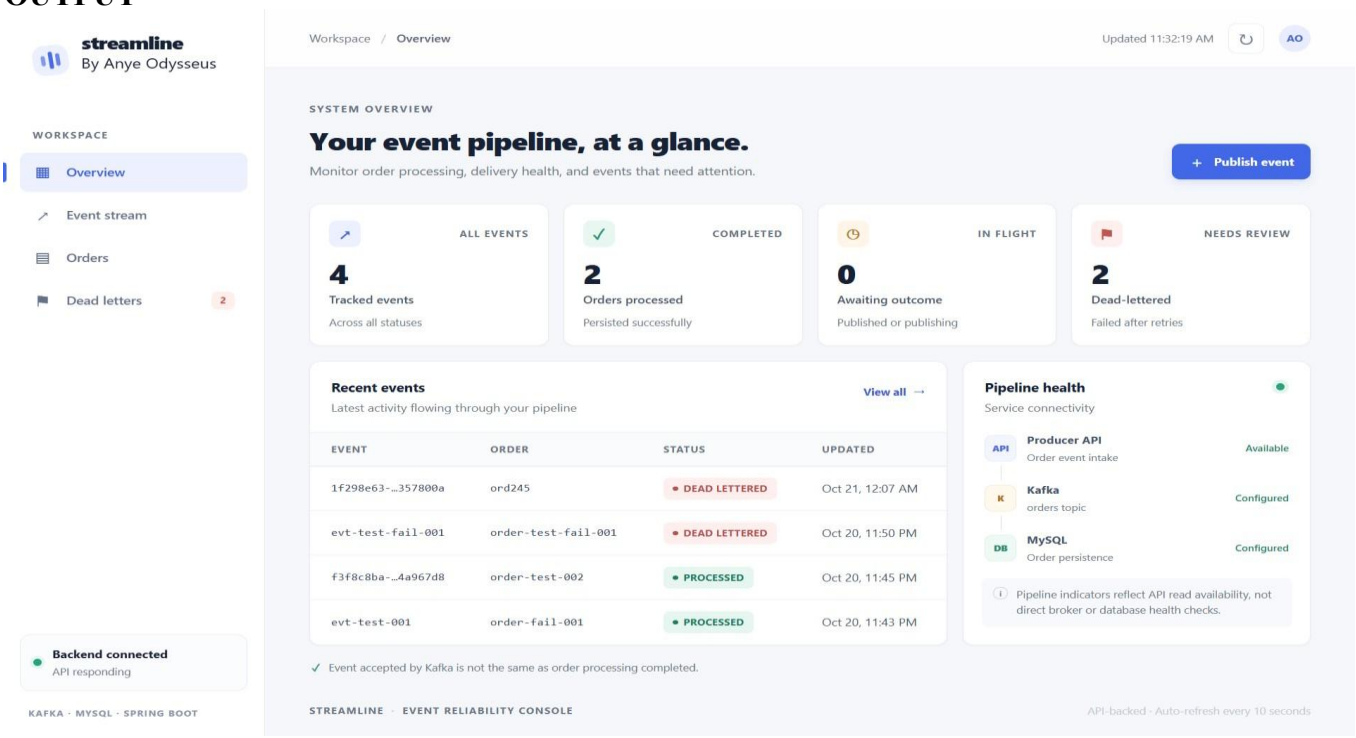


Fig 8.1: monitoring, order processing, delivery health and events that need attention

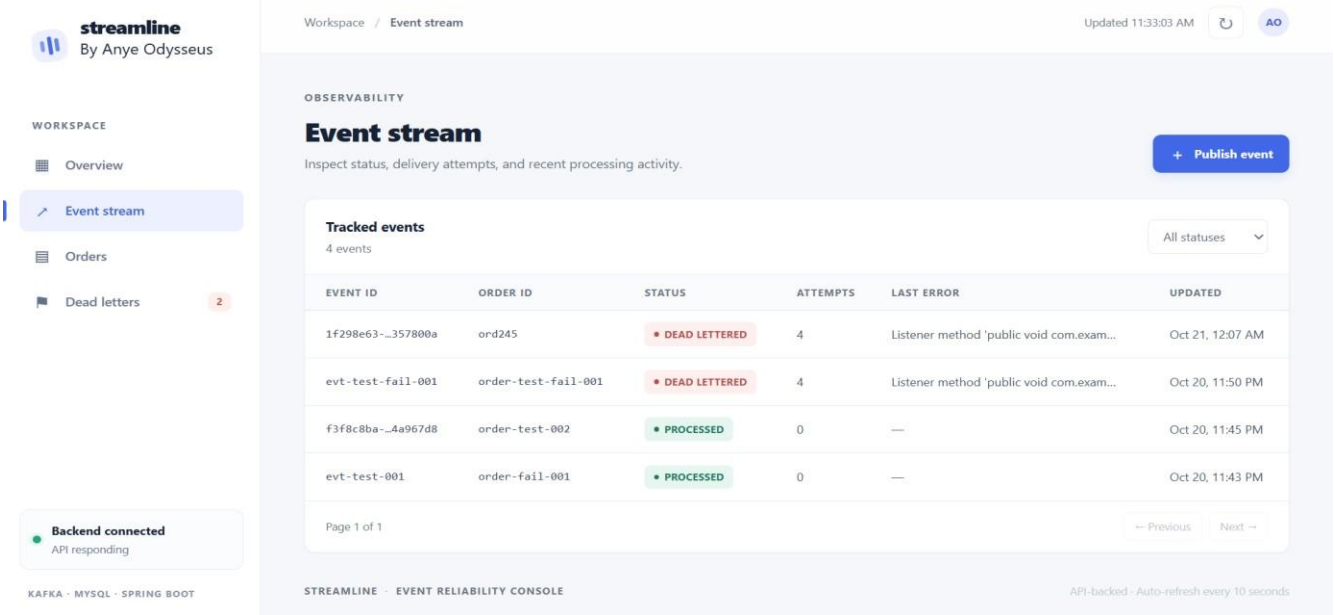


Fig 8.2: all events and orders

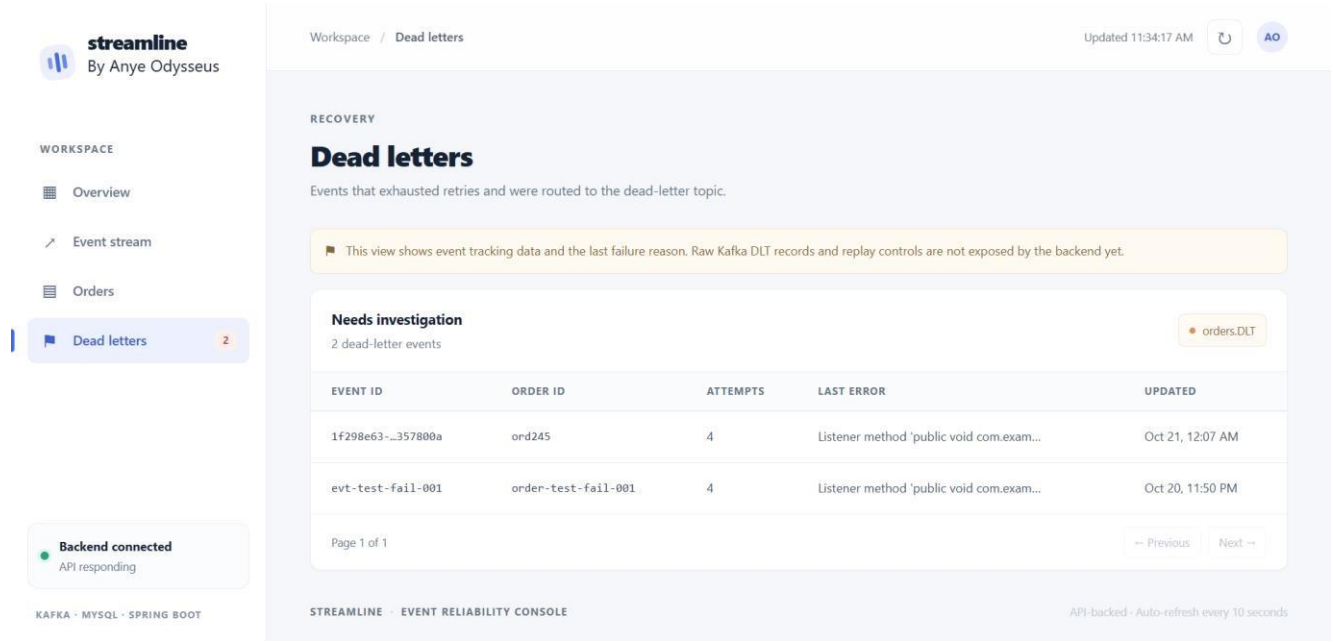


Fig 8.2: dead letter heads

8) LEARNING OUTCOMES

After completing this experiment, I learned to:

- Explain Kafka's role in decoupling an order-producing REST API from asynchronous order processing.
- Configure and run Kafka locally using Docker Compose and connect a Spring Boot application to the broker.
- Publish and consume JSON events using Spring Kafka producers, consumers, serializers, and deserializers.
- Implement idempotent processing so duplicate Kafka deliveries do not create duplicate orders.
- Apply database transactions to keep event processing and order persistence consistent when failures occur.
- Use a dead-letter topic (DLT) to isolate and inspect messages that still fail after retries.
- Configure retry strategies with exponential backoff to handle temporary processing errors.